

Unit-1

1)A company stores employee IDs in ascending order. Write a C program that accepts n employee IDs, searches for a required ID using Binary Search, displays its position when found, reports when it is absent, and counts the number of comparisons. Test the program for both successful and unsuccessful searches

Source code:-

```
#include <stdio.h>

int main() {
    int n, key, low, high, mid, c = 0, found = 0;
    printf("Enter number of employees: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter employee IDs in ascending order:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    printf("Enter ID to search: ");
    scanf("%d", &key);
    low = 0;
    high = n - 1;
    while(low <= high) {
        c++;
        mid = (low + high) / 2;
        if(a[mid] == key) {
            printf("Employee ID found at position: %d\n", mid + 1);
            found = 1;
            break;
        }
        else if(a[mid] < key) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }
    if(!found) {
        printf("Employee ID %d is absent from the records.\n", key);
    }
    printf("Total comparisons: %d\n", c);
    return 0;
}
```

2)A teacher wants to arrange student marks in ascending order and also measure how much rearrangement is necessary. Write a C program using Insertion Sort that accepts n marks, displays the array after every pass, counts the total number of element shifts, and displays the final sorted list and shift count.

Source code:

```
#include <stdio.h>
int main() {
    int n, key, j, c = 0;
    printf("Enter number of students: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter student marks:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    for(int i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while(j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            c++;
            j--;
        }
        a[j + 1] = key;
        printf("Pass %d: ", i);
        for(int k = 0; k < n; k++) {
            printf("%d ", a[k]);
        }
        printf("\n");
    }
    printf("Final sorted marks: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\nTotal comparisions: %d", c);
    return 0;
}
```

Unit-2

3)An expression-processing application receives an arithmetic expression in infix form. Write a C program using a stack to convert it to postfix form while correctly handling parentheses and operator precedence for +, -, *, / and ^. Test the program using an expression containing multiple operators and parentheses.

Source code:

```
#include <stdio.h>
#include <ctype.h>
char stack[100];
int top = -1;
void push(char c) {
    stack[++top] = c;
}
char pop() {
    if(top == -1) return -1;
    return stack[top--];
}
int prec(char c) {
    if(c == '^')
        return 3;
    if(c == '*' || c == '/')
        return 2;
    if(c == '+' || c == '-')
        return 1;
    return 0;
}
int main() {
    char exp[100], pst[100];
    int k = 0;
    printf("Enter infix expression: ");
    scanf("%s", exp);
    for(int i = 0; exp[i] != '\0'; i++) {
        char ch = exp[i];
        if(isalnum(ch)) {
            pst[k++] = ch;
        }
        else if(ch == '(') {
            push(ch);
        }
        else if(ch == ')') {
            while(top != -1 && stack[top] != '(') {
                pst[k++] = pop();
            }
            pop();
        }
    }
}
```

```

        else {
            while(top != -1 && stack[top] != '(' && (prec(stack[top]) > prec(ch) || (prec(stack[top]) ==
prec(ch) && ch != '^'))) {
                pst[k++] = pop();
            }
            push(ch);
        }
    }
    while(top != -1) {
        pst[k++] = pop();
    }
    pst[k] = '\0';
    printf("postfix expression: %s\n", pst);
    return 0;
}

```

4)A service centre uses a fixed-size request buffer in which released positions must be reused. Write a C program to implement a Circular Queue using an array with insertion, deletion, display, overflow and underflow operations. Demonstrate that positions freed after deletion can be reused for new requests.

Source code:

```

#include <stdio.h>
#define SIZE 5
int q[SIZE], front = -1, rear = -1;
void enqueue(int id) {
    if((front == 0 && rear == SIZE - 1) || (front == rear + 1)) {
        printf("Queue Overflow! Request buffer is full.\n");
        return;
    }
    if(front == -1) {
        front = rear = 0;
    }
    else if(rear == SIZE - 1 && front != 0) {
        rear = 0;
    }
    else {
        rear++;
    }
    q[rear] = id;
    printf("Request %d inserted at slot %d\n", id, rear);
}
void dequeue() {
    if(front == -1) {
        printf("Queue Underflow! Buffer is empty.\n");
        return;
    }
}

```

```

    }
    printf("Request %d processed and removed from slot %d\n", q[front], front);
    if(front == rear) {
        front = rear = -1;
    }
    else if(front == SIZE - 1) {
        front = 0;
    }
    else {
        front++;
    }
}

void display() {
    if(front == -1) {
        printf("Request buffer is empty.\n");
        return;
    }
    printf("Current Buffer: ");
    if(rear >= front) {
        for(int i = front; i <= rear; i++) {
            printf("[%d: %d] ", i, q[i]);
        }
    }
    else {
        for(int i = front; i < SIZE; i++) {
            printf("[%d: %d] ", i, q[i]);
        }
        for(int i = 0; i <= rear; i++) {
            printf("[%d: %d] ", i, q[i]);
        }
    }
    printf("\n");
}

int main() {
    int choice, val;
    while(1) {
        printf("1.Insert Request 2.Process Request 3.Display Buffer 4.Exit: ");
        scanf("%d", &choice);
        if(choice == 1) {
            printf("Enter Request ID: ");
            scanf("%d", &val);
            enqueue(val);
        }
        else if(choice == 2) {

```

```

        dequeue();
    }
    else if(choice == 3) {
        display();
    }
    else if(choice == 4) {
        break;
    }
    else {
        printf("Invalid selection!\n");
    }
}
return 0;
}

```

Unit-3

5)A department maintains student roll numbers dynamically. Write a C program using a Singly Linked List to create the list, insert at the beginning and end, search for a specified roll number, delete a specified roll number, and display the updated list after each operation. Handle the case when a requested roll number is not available.

Source code:

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int roll;
    struct Node* next;
};
struct Node* head = NULL;
void display() {
    if(head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node* temp = head;
    printf("Roll Numbers: ");
    while(temp != NULL) {
        printf("%d -> ", temp->roll);
        temp = temp->next;
    }
    printf("NULL\n");
}
void insertBeginning(int val) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));

```

```

    n->roll = val;
    n->next = head;
    head = n;
    display();
}
void insertEnd(int val) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->roll = val;
    n->next = NULL;
    if(head == NULL) {
        head = n;
    }
    else {
        struct Node* temp = head;
        while(temp->next != NULL) temp = temp->next;
        temp->next = n;
    }
    display();
}
void search(int val) {
    struct Node* temp = head;
    int pos = 1, found = 0;
    while(temp != NULL) {
        if(temp->roll == val) {
            printf("Roll number %d found at position %d\n", val, pos);
            found = 1;
            break;
        }
        temp = temp->next;
        pos++;
    }
    if(!found) printf("Roll number %d is not available in the list.\n", val);
}
void deleteNode(int val) {
    if(head == NULL) {
        printf("List is empty, roll number not found.\n");
        return;
    }
    struct Node *temp = head, *prev = NULL;
    if(temp->roll == val) {
        head = temp->next;
        free(temp);
        printf("Deleted %d successfully.\n", val);
        display();
    }

```

```

        return;
    }
    while(temp != NULL && temp->roll != val) {
        prev = temp;
        temp = temp->next;
    }
    if(temp == NULL) {
        printf("Roll number %d is not available in the list.\n", val);
        return;
    }
    prev->next = temp->next;
    free(temp);
    printf("Deleted %d successfully.\n", val);
    display();
}

int main() {
    int choice, val;
    while(1) {
        printf("1.Insert Beg 2.Insert End 3.Search 4.Delete 5.Display 6.Exit: ");
        scanf("%d", &choice);
        if(choice == 1) {
            printf("Enter roll number: ");
            scanf("%d", &val);
            insertBeginning(val);
        }
        else if(choice == 2) {
            printf("Enter roll number: ");
            scanf("%d", &val);
            insertEnd(val);
        }
        else if(choice == 3) {
            printf("Enter roll number to search: ");
            scanf("%d", &val);
            search(val);
        }
        else if(choice == 4) {
            printf("Enter roll number to delete: ");
            scanf("%d", &val);
            deleteNode(val);
        }
        else if(choice == 5) {
            display();
        }
        else if(choice == 6) {

```

```

        break;
    }
    else {
        printf("Invalid choice!\n");
    }
}
return 0;
}

```

6) Develop a C program for a Doubly Linked List representing a sequence of web pages visited by a user. The program should insert a new page, move forward and backward, delete a specified page, and display the pages from first-to-last and last-to-first while handling beginning and end conditions correctly.

Source code:

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int page;
    struct Node* prev;
    struct Node* next;
};
struct Node* head = NULL;
struct Node* curr = NULL;
void insertPage(int id) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->page = id;
    n->next = NULL;
    n->prev = NULL;
    if(head == NULL) {
        head = n;
        curr = head;
    }
    else {
        struct Node* temp = head;
        while(temp->next != NULL) temp = temp->next;
        temp->next = n;
        n->prev = temp;
        curr = n;
    }
    printf("Visited page %d\n", id);
}
void moveForward() {
    if(curr != NULL && curr->next != NULL) {
        curr = curr->next;
        printf("Moved forward to page %d\n", curr->page);
    }
}

```

```

    }
    else {
        printf("Already at the latest page. Forward not possible.\n");
    }
}

void moveBackward() {
    if(curr != NULL && curr->prev != NULL) {
        curr = curr->prev;
        printf("Moved backward to page %d\n", curr->page);
    }
    else {
        printf("Already at the first page. Backward not possible.\n");
    }
}

void displayForward() {
    if(head == NULL) {
        printf("No browsing history.\n");
        return;
    }
    struct Node* temp = head;
    printf("Pages (First-to-Last): ");
    while(temp != NULL) {
        printf("%d ", temp->page);
        temp = temp->next;
    }
    printf("\n");
}

void displayBackward() {
    if(head == NULL) {
        printf("No browsing history.\n");
        return;
    }
    struct Node* temp = head;
    while(temp->next != NULL) temp = temp->next;
    printf("Pages (Last-to-First): ");
    while(temp != NULL) {
        printf("%d ", temp->page);
        temp = temp->prev;
    }
    printf("\n");
}

void deletePage(int id) {
    if(head == NULL) {
        printf("History empty, cannot delete.\n");
    }
}

```

```

        return;
    }
    struct Node* temp = head;
    while(temp != NULL && temp->page != id) temp = temp->next;
    if(temp == NULL) {
        printf("Page %d not found in history.\n", id);
        return;
    }
    if(temp == head)
        head = temp->next;
    if(temp->prev != NULL)
        temp->prev->next = temp->next;
    if(temp->next != NULL)
        temp->next->prev = temp->prev;
    if(curr == temp)
        curr = (temp->next != NULL) ? temp->next : temp->prev;
    free(temp);
    printf("Page %d deleted from history.\n", id);
}

int main() {
    int choice, id;
    while(1) {
        printf("1.Visit Page 2.Forward 3.Backward 4.Display Fwd 5.Display Back 6.Delete Page\n7.Exit: ");
        scanf("%d", &choice);
        if(choice == 1) {
            printf("Enter Page ID: ");
            scanf("%d", &id);
            insertPage(id);
        }
        else if(choice == 2) {
            moveForward();
        }
        else if(choice == 3) {
            moveBackward();
        }
        else if(choice == 4) {
            displayForward();
        }
        else if(choice == 5) {
            displayBackward();
        }
        else if(choice == 6) {
            printf("Enter Page ID to delete: ");

```

```

        scanf("%d", &id);
        deletePage(id);
    }
    else if(choice == 7) {
        break;
    }
    else {
        printf("Invalid selection!\n");
    }
}
return 0;
}

```

Unit-4

7)A system stores unique integer identification numbers using a Binary Search Tree. Write a C program to insert n values, display inorder, preorder and postorder traversals, search for a specified value, and report whether it exists. Use the output to explain why inorder traversal produces sorted values

Source code:

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int id;
    struct Node *left, *right;
};
struct Node* createNode(int val) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->id = val;
    n->left = n->right = NULL;
    return n;
}
struct Node* insert(struct Node* root, int val) {
    if(root == NULL)
        return createNode(val);
    if(val < root->id)
        root->left = insert(root->left, val);
    else if(val > root->id)
        root->right = insert(root->right, val);
    return root;
}
void inorder(struct Node* root) {
    if(root != NULL) {
        inorder(root->left);

```

```

        printf("%d ", root->id);
        inorder(root->right);
    }
}

void preorder(struct Node* root) {
    if(root != NULL) {
        printf("%d ", root->id);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node* root) {
    if(root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->id);
    }
}

int search(struct Node* root, int val) {
    if(root == NULL)
        return 0;
    if(root->id == val)
        return 1;
    if(val < root->id)
        return search(root->left, val);
    return search(root->right, val);
}

int main() {
    struct Node* root = NULL;
    int n, val, key;
    printf("Enter number of identification numbers: ");
    scanf("%d", &n);
    printf("Enter unique IDs:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d", &val);
        root = insert(root, val);
    }
    printf("\nInorder Traversal: ");
    inorder(root);
    printf("\nPreorder Traversal: ");
    preorder(root);
    printf("\nPostorder Traversal: ");
    postorder(root);
    printf("\n\nEnter ID to search: ");

```

```

scanf("%d", &key);
if(search(root, key)) {
    printf("ID %d exists in the BST.\n", key);
}
else {
    printf("ID %d does not exist in the BST.\n", key);
}
printf("\nExplanation: Inorder traversal visits (Left Subtree -> Root -> Right Subtree). Because
every node in the left subtree is smaller than root and every node in the right subtree is larger,
this recursive ordering outputs all values strictly sorted in ascending order.\n");
return 0;
}

```

8) Extend a Binary Search Tree program to support deletion. The program should create a BST, delete a user-specified node, correctly handle nodes with zero, one and two children, and display inorder traversal before and after deletion. Test the program separately for all three deletion cases

Source code:

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *left, *right;
};
struct Node* createNode(int val) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = val;
    n->left = n->right = NULL;
    return n;
}
struct Node* insert(struct Node* root, int val) {
    if(root == NULL)
        return createNode(val);
    if(val < root->data)
        root->left = insert(root->left, val);
    else if(val > root->data)
        root->right = insert(root->right, val);
    return root;
}
struct Node* findMin(struct Node* root) {
    while(root->left != NULL)
        root = root->left;
    return root;
}
struct Node* deleteNode(struct Node* root, int val, int *deleted) {

```

```

if(root == NULL)
return NULL;
if(val < root->data) {
    root->left = deleteNode(root->left, val, deleted);
}
else if(val > root->data) {
    root->right = deleteNode(root->right, val, deleted);
}
else {
    *deleted = 1;
    if(root->left == NULL && root->right == NULL) {
        printf("[Case 0: Leaf node deleted]\n");
        free(root);
        return NULL;
    }
    else if(root->left == NULL) {
        printf("[Case 1: Node with only right child deleted]\n");
        struct Node* temp = root->right;
        free(root);
        return temp;
    }
    else if(root->right == NULL) {
        printf("[Case 1: Node with only left child deleted]\n");
        struct Node* temp = root->left;
        free(root);
        return temp;
    }
    else {
        printf("[Case 2: Node with two children replaced by inorder successor]\n");
        struct Node* temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data, deleted);
    }
}
}
return root;
}

void inorder(struct Node* root) {
    if(root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {

```

```

struct Node* root = NULL;
int n, val, delVal;
printf("Enter number of nodes: ");
scanf("%d", &n);
printf("Enter values to insert:\n");
for(int i = 0; i < n; i++) {
    scanf("%d", &val);
    root = insert(root, val);
}
printf("Inorder before deletion: ");
inorder(root);
printf("\n");
while(1) {
    printf("\nEnter value to delete (-1 to exit): ");
    scanf("%d", &delVal);
    if(delVal == -1) break;
    int deleted = 0;
    printf("Inorder before: ");
    inorder(root);
    printf("\n");
    root = deleteNode(root, delVal, &deleted);
    if(!deleted) {
        printf("Node %d not found in the BST.\n", delVal);
    }
    printf("Inorder after: ");
    inorder(root);
    printf("\n");
}
return 0;
}

```